

RUST

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ *Red star = Important Topic (read carefully, exam / interview favorite)*

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to Rust

★ 2. Variables and Mutability

- 3. Data Types

★ 4. Ownership

★ 5. Borrowing and References

- 6. Functions
- 7. Conditional Statements (if, else if, else)
- 8. Loops - loop, while, for

★ 9. Structs

- 10. Enums
- 11. Vectors
- 12. String Type

★ 13. Error Handling - Result and Option

- 14. Traits

★ 1. Introduction to Rust [IMPORTANT TOPIC]

Explanation:

Rust is a modern programming language known for being extremely fast AND extremely safe. Unlike C or C++, Rust catches most memory-related bugs (a common cause of crashes and security holes) at compile time, before the program even runs.

Key Points:

- As fast as C/C++, but much safer.
- No 'garbage collector' needed, thanks to its unique Ownership system.
- Popular for system programming, game engines, and web assembly.

Example Code:

```
fn main() {  
    println!("Hello, World!");  
}
```

Output:

Hello, World!

 *Note: The ! after println is not a mistake — it shows that println! is a 'macro', not a normal function.*

★ 2. Variables and Mutability *[IMPORTANT TOPIC]*

Explanation:

In Rust, variables are IMMUTABLE (cannot be changed) by default — this is different from most other languages. To allow a variable to change, we must specifically mark it with the word 'mut'.

Example Code:

```
let name = "Pravendra";    // immutable by default
let mut age = 21;          // mutable, can be changed

age = 22;
println!("{}", name, age);
```

Output:

```
Pravendra is 22 years old
```

💡 *Note: This 'immutable by default' rule is one of Rust's core safety features — it prevents accidental changes to data.*

3. Data Types

Explanation:

Rust has clear, strict data types, and it can usually detect a variable's type automatically without us writing it directly.

Key Points:

- i32 -> a whole number (integer)
- f64 -> a decimal number (float)
- bool -> true or false
- char -> a single character
- String / &str -> text

Example Code:

```
let age: i32 = 21;  
let height: f64 = 5.9;  
let is_student: bool = true;  
println!("{}", age);
```

Output:

```
21
```

★ 4. Ownership *[IMPORTANT TOPIC]*

Explanation:

Ownership is Rust's most important and unique concept. Every value in Rust has exactly ONE 'owner' variable. When that owner variable goes out of use, Rust automatically cleans up the memory — no manual cleanup (like C's `free()`) is needed, and no memory leaks or crashes happen because of it.

Example Code:

```
let s1 = String::from("hello");  
let s2 = s1;    // ownership MOVES from s1 to s2  
  
// println!("{}", s1);    // ERROR - s1 is no longer valid here  
println!("{}", s2);        // this works fine
```

Output:
hello

 *Note: This 'move' behavior looks strict at first, but it is exactly what prevents whole categories of memory bugs common in other languages.*

★ 5. Borrowing and References *[IMPORTANT TOPIC]*

Explanation:

Instead of always transferring ownership, Rust lets us 'borrow' a value using a reference (&), so a function can use a value WITHOUT taking ownership of it.

Example Code:

```
fn calculate_length(s: &String) -> usize {
    s.len()
}

fn main() {
    let s1 = String::from("hello");
    let len = calculate_length(&s1);    // borrowing, not moving
    println!("Length of '{}' is {}", s1, len);
}
```

Output:

```
Length of 'hello' is 5
```

6. Functions

Explanation:

A function is a reusable block of code, defined using the 'fn' keyword.

Example Code:

```
fn add_numbers(a: i32, b: i32) -> i32 {  
    a + b    // no 'return' or semicolon needed for the last line  
}  
  
fn main() {  
    let total = add_numbers(5, 7);  
    println!("Total: {}", total);  
}
```

Output:

Total: 12

7. Conditional Statements (if, else if, else)

Explanation:

Conditional statements let the program make decisions based on whether something is true or false.

Example Code:

```
let marks = 75;

if marks >= 90 {
    println!("Grade A");
} else if marks >= 60 {
    println!("Grade B");
} else {
    println!("Grade C");
}
```

Output:

Grade B

8. Loops - loop, while, for

Explanation:

Rust has three kinds of loops: 'loop' (repeats forever until told to stop), 'while' (repeats while a condition is true), and 'for' (repeats over a range or collection).

Example Code:

```
for i in 1..4 {  
    println!("Count: {}", i);  
}
```

Output:

```
Count: 1  
Count: 2  
Count: 3
```

 *Note: The range 1..4 includes 1, 2, 3 — the last number is NOT included, similar to Python.*

★ 9. Structs *[IMPORTANT TOPIC]*

Explanation:

A struct groups related data together under one name. Rust does not have traditional classes — structs, along with 'impl' blocks (to add functions), are Rust's way of organizing data and behavior.

Example Code:

```
struct Student {  
    name: String,  
    age: i32,  
}  
  
let s1 = Student { name: String::from("Pravendra"), age: 21 };  
println!("{}", s1.name, s1.age);
```

Output:

```
Pravendra is 21
```

10. Enums

Explanation:

An enum lets us define a type that can be ONE of several different named values. Rust's enums are more powerful than in many languages — each value can even hold its own extra data.

Example Code:

```
enum Direction {  
    North,  
    South,  
}  
  
let my_dir = Direction::North;  
  
match my_dir {  
    Direction::North => println!("Heading North"),  
    Direction::South => println!("Heading South"),  
}
```

Output:

```
Heading North
```

11. Vectors

Explanation:

A Vector (Vec) is Rust's version of a dynamic, growable list — similar to an ArrayList in Java or a list in Python.

Example Code:

```
let mut numbers: Vec<i32> = vec![10, 20, 30];
numbers.push(40);

println!("{:?}", numbers);
println!("{}", numbers[0]);
```

Output:

```
[10, 20, 30, 40]
10
```

 *Note: {:?} is used inside println! to print a whole collection like a vector in a readable format.*

12. String Type

Explanation:

Rust actually has two main string types: 'String' (owned, can grow and change) and '&str' (a fixed, borrowed view of text). Understanding this difference is an important part of learning Rust.

Example Code:

```
let mut greeting = String::from("Hello");  
greeting.push_str(", Rust!");  
println!("{}", greeting);
```

Output:

```
Hello, Rust!
```

★ 13. Error Handling - Result and Option *[IMPORTANT TOPIC]*

Explanation:

Rust does not use exceptions like try-catch. Instead, it uses two special types: 'Option' (represents a value that might or might not exist) and 'Result' (represents a task that might succeed or fail), forcing the programmer to properly handle every possible case.

Example Code:

```
fn divide(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        return Err(String::from("Cannot divide by zero!"));
    }
    Ok(a / b)
}

match divide(10, 0) {
    Ok(result) => println!("Result: {}", result),
    Err(msg) => println!("Error: {}", msg),
}
```

Output:

Error: Cannot divide by zero!

14. Traits

Explanation:

A trait defines a set of behaviors (methods) that a type can implement. It is similar to an 'interface' in other languages, and is Rust's way of sharing common behavior across different types.

Example Code:

```
trait Greet {  
    fn say_hello(&self);  
}  
  
struct Person;  
  
impl Greet for Person {  
    fn say_hello(&self) {  
        println!("Hello!");  
    }  
}  
  
let p = Person;  
p.say_hello();
```

Output:

```
Hello!
```